LIFE OS

Rationales & Design Evolution

Reasoning genealogy - Checkpoint v0.1

Purpose	Preserve the reasoning structures, trade-offs and case evidence that produced the current design posture, without turning the conversation into a transcript.

Status	Mixed: Established conclusions + Working boundaries
Reading mode	Deep reasoning / genealogy
Canonical sources	Master Extraction v0.1; Models 03-08; Case Studies 09; Design Log 11
Checkpoint boundary	Pre-Ontology Minimum v0.1

This document explains why the checkpoint looks the way it does. Future versions may revise conclusions, but should not erase the path that made them reasonable at this point.

0. How to read the Rationales

ESTABLISHED	These chapters preserve the reasoning structure that produced the current checkpoint. They are not transcripts and they do not elevate Working material to Established.

Each rationale follows the same chain: problem -> observed cases -> alternatives/tensions -> emerging insight -> consequence -> remaining uncertainty. The goal is reconstructability: months later, a reader should be able to understand why a construct exists and which real problems it was meant to solve.

0.1 Cross-rationale dependency map

Several insights reinforce each other rather than standing alone. Progressive Structuring reduces premature taxonomy; Derive, do not duplicate reduces maintenance cost; Views as Projections keeps representation singular; System as Mirror limits automation power; Capability-driven operationality lets useful behavior emerge from partial schema.

```
Progressive Structuring -> Entity promotion -> Emergent schema -> Capability-driven operations
Relational Knowledge -> Generic/contextual separation -> Derivation -> Reuse across projects System
as Mirror -> Rails -> Planned vs Actual -> Revisable Self-model
```

R01 - Progressive Structuring

Problem. A fixed schema imposed at capture time creates friction exactly when information is least mature.

Observed cases

- App Mobile started as a phrase and only later deserved reusable independent structure.
- A crochet idea can start as a screenshot plus a sentence and gain components only when the vision clarifies.
- A Goal can begin as simple text and later acquire completion criteria, subgoals and relations.

Emerging insight

CURRENT CONCLUSION	The system should allow a path from loose text to simple value, structured value, Entity and relational network without requiring that every item travel the full path.

Consequences

Promotion is user-controlled; the system may suggest that structure has become useful, but it should not silently change semantic identity.

Remaining uncertainty

OPEN	The technical boundary between structured value and Entity remains Open.

R02 - Knowledge as a Relational Graph

Problem. Knowledge does not naturally fit a single tree: the same concept participates in many methods, projects and explanatory structures.

Observed cases

- F2L belongs to the knowledge of multiple cube-solving methods and should not be duplicated.

- GDPR and mobile-app obligations should be reusable across concrete apps.
- Crochet structures such as a base shape can be referenced from multiple projects.

Emerging insight

CURRENT CONCLUSION	Treat Concept as a candidate Entity type and let Knowledge emerge from Concept + content + semantic relations rather than from duplicated containers.

Consequences

The same Concept can be reached through different lenses without changing its canonical identity.

Remaining uncertainty

OPEN	The exact boundary between Concept and Knowledge remains Working.

R03 - Generic Knowledge vs Contextual Application

Problem. A generic concept can have project-specific relevance, status or blocking state without that state being intrinsic to the concept.

Observed cases

- App Mobile generally requires attention to regulations; Solducci is one concrete app.
- GDPR is not globally blocked or addressed; the Solducci<->GDPR application can be blocked, partial or resolved.

Emerging insight

CURRENT CONCLUSION	Keep generic Knowledge context-independent. Put project-specific state on the relation/application between the concrete entity and generic knowledge.

Consequences

This preserves reuse and prevents one project from contaminating another project's interpretation of the same knowledge.

Remaining uncertainty

OPEN	How rich relation metadata should become is Open.

R04 - Why Project Needs Identity

Problem. Projects must survive interruption, evolve structurally and accumulate decisions, decomposition and reusable outputs.

Observed cases

- Solducci has a vision that changes while preserving continuity.
- LaGenteStrana emerged from informal experiments into a persistent collaborative initiative.
- Agrivoltaico may remain inactive for long periods yet still needs resumable state.

Emerging insight

CURRENT CONCLUSION	Project deserves Entity-level identity because it has independent lifecycle, operational memory, relations, structure and resume needs.

Consequences

A Project can remain lightweight early and gain operational properties as required.

Remaining uncertainty

OPEN	The minimal property set for Project should not be fixed before the ontology stress test.

R05 - Session as Experience Sampling

Problem. Exhaustive life logging creates more maintenance than insight, but no sampling loses evidence needed for reflection and learning.

Observed cases

- Speedcubing benefits from session summaries without storing every solve.
- Development and crochet sessions can preserve what happened, observations and outputs.
- A ceremony also samples a moment of deliberate self/system maintenance.

Emerging insight

CURRENT CONCLUSION	Session is a Working construct for sampling lived activity/experience at a useful granularity.

Consequences

Metrics, observations, feelings and outputs are optional additions whose presence enables different analyses.

Remaining uncertainty

OPEN	Exact Session ontology and its relation to generic temporal constructs remain Working.

R06 - Phase vs Temporal Period

Problem. The word phase was being used both for a project state and for a planning window, creating semantic ambiguity.

Observed cases

- "Develop Context Manager" may cross multiple months.
- A September-October planning period can contain several project phases.

Emerging insight

CURRENT CONCLUSION	Separate state/focus from when: Phase describes configuration, stage or focus; Temporal Period describes an interval used for aggregation/planning.

Consequences

The two can be related without being identical.

Remaining uncertainty

OPEN	Phase representation itself - Entity, structured state or contextual construct - remains Open.

R07 - Rails instead of Rigid Scheduling

Problem. A detailed calendar ignores changing energy, context and desire and turns planning into control overhead.

Observed cases

- Weekly planning knows available time, places, tools and likely energy.
- During a free slot the person wants compatible options already filtered by prior priorities, then chooses based on current state.

Emerging insight

CURRENT CONCLUSION	Plan constrained choice rather than exact execution: define commitments and compatible rails, then choose the specific action at runtime.

Consequences

This preserves intentional balance without removing autonomy.

Remaining uncertainty

OPEN	The precise formal output of weekly ceremonies remains Open.

R08 - System as Mirror

Problem. Automation that silently decides importance, identity or commitment would undermine the core purpose of increasing awareness and autonomy.

Observed cases

- The system can notice an Idea behaving like a Project.
- It can surface long inactivity, repeated observations or a mismatch between declared allocation and actual activity.
- It can notice recurring value language without declaring those to be the person's true values.

Emerging insight

CURRENT CONCLUSION	Derive signals, expose evidence and invite interpretation. Suggestions can propose structure or reflection, but final semantic/intentional changes belong to the user.

Consequences

Explainability is part of usefulness: the user should be able to understand why a signal appeared.

Remaining uncertainty

OPEN	Threshold design and anti-annoyance mechanisms remain future work.

R09 - Self, Values & Resonance

Problem. A planning system that ignores what matters personally becomes shallow, while a rigid value taxonomy can itself become controlling.

Observed cases

- Autonomy repeatedly appeared across software, creativity and life design.
- Understanding was described as using local domains as lenses on the whole.
- Human connection has intrinsic value and also expands perspective.
- Positive impact is associated with empowerment rather than mere profitable need capture.

Emerging insight

CURRENT CONCLUSION	Represent the Self lightly and revisionably. Values can be Concepts; resonance/interest can be relations whose importance may change over time.

Consequences

Ceremonies can compare declared self-model, observed behavior and subjective experience without treating any one as final truth.

Remaining uncertainty

OPEN	The ontological status of Self and the temporal behavior of resonance are Open.

R10 - Schema -> Capability -> Operations

Problem. Rigid types force irrelevant fields onto entities just to unlock useful behaviors.

Observed cases

- A Task-like entity may need status transitions but not scheduling.
- Another entity may become schedulable by gaining temporal properties.
- A Session can gain quantitative analysis only when Metric data exists.

Emerging insight

CURRENT CONCLUSION	Define operations as enabled by sufficient properties/relations rather than by all-or-nothing class membership.

Consequences

This supports gradual operationality and keeps schemas proportional to actual need.

Remaining uncertainty

OPEN	Formal capability declaration, inference and UI exposure are Open.

R11 - Derive, Do Not Duplicate

Problem. Stored redundant facts drift apart and increase maintenance cost.

Observed cases

- If a Session uses Friedrich and Friedrich contains F2L, storing Session->F2L may be unnecessary.
- Knowledge completion can often be derived from subparts relative to the current structure.
- Balance and inactivity signals should be computed from underlying events/relations when possible.

Emerging insight

CURRENT CONCLUSION	Prefer canonical facts plus reliable derivation. Materialize only when derivation is expensive, ambiguous or historically meaningful.

Consequences

Derived values should remain explainable and tied to the current structure/version of the thing being measured.

Remaining uncertainty

OPEN	Materialization policy is implementation-level and intentionally deferred.

R12 - Views as Projections

Problem. Different contexts need different information density, but duplicating state for each context creates inconsistency.

Observed cases

- A recently active project needs an execution view; an inactive one needs context reconstruction.
- Exploration emphasizes unknowns/resources/scenarios; execution emphasizes tasks/blockers.
- Ceremonies need cross-cutting planned-vs-actual views.

Emerging insight

CURRENT CONCLUSION	Keep one underlying structure and project it through lenses controlled by focus, temporal distance, mode and inactivity.

Consequences

A signal changes what may deserve attention; a view changes how existing information is presented.

Remaining uncertainty

OPEN	View selection and personalization are future interaction design questions.

13. Synthesis: the design posture produced by these Rationales

Taken together, the Rationales describe a system that starts permissive, becomes structured only when useful, keeps facts canonical, derives secondary state, separates generic meaning from context, samples life rather than logging it exhaustively, and preserves the user as the final interpreter of intention and identity.

Design tension	Current posture
Freedom vs structure	Start loose; add structure when operations, reuse or resume justify it.
Automation vs autonomy	Detect and explain; suggest; do not silently decide.
Reuse vs context	Keep generic Concepts canonical; contextualize on relations/applications.
Planning vs spontaneity	Commit to rails and proportions; choose concrete action in context.
Memory vs maintenance	Preserve operational/structural memory; avoid exhaustive snapshots/logging.
Identity vs evolution	Maintain a revisable self-model rather than a fixed taxonomy.